

Soma: Achieving Low-Level Performance in a General-Purpose Dependently Typed Functional Language via Interaction Nets

Gabriel Di Lucca Minatel

Abstract

Historically, functional programming languages have struggled to match the low-level performance of imperative languages due to their high-level abstractions and runtime overheads. This paper introduces Soma, a general-purpose dependently typed functional programming language that leverages interaction nets to optimize performance while maintaining strong type safety and expressiveness.

1. Introduction

Programming languages can be broadly categorized into imperative and functional paradigms, inspired by two different models of computation: the Turing machine and the lambda calculus, respectively.

Functional programming languages, while offering powerful abstractions and strong type systems, often face challenges in achieving low-level performance comparable to imperative languages. On the other hand, imperative languages, with their mutable state and control flow constructs, can be optimized for performance but may lack the elegance, expressiveness and safety features of functional languages.

To address this challenge, we present Soma, a dependently typed functional programming language that utilizes interaction nets as its underlying computational model. Interaction nets provide a graphical representation of computation that allows for efficient reduction strategies, enabling Soma to achieve low-level performance comparable to imperative languages.

2. Interaction Nets Overview

After linear logic was theorized, several researchers realized it could model functional computation with the bonus of fine-grained resource management. This led to variants of lambda calculus and the development of Interaction Nets by Yves Lafont in 1990.

Interaction nets are a form of graph rewriting system where computation is represented as the interaction between nodes (or agents) connected by edges. Each node represents a computational entity and the edges represent the flow of data between these entities.

2.1. Interaction Combinators

Interaction combinators are a minimal set of interaction net agents that can simulate any interaction net. They consist of three types of agents: the δ (duplicator) agent, the γ (constructor) agent and the ε (eraser) agent. These agents interact according to specific rules that define how they can be rewritten when they come into contact with each other.

These follow simple interaction rules: annihilation, commutation and erasure. Annihilation occurs when two agents of the same type collide, resulting in their removal from the net. Commutation happens when a constructor meets a duplicator. The duplicator clones the constructor and the constructor splits the duplicator. This is how copying propagates through a data structure. Finally, erasure happens when an eraser meets any agent, destroying it and its auxiliary ports.

The Interaction Combinators are Turing complete, meaning they can simulate any Turing machine. This property makes them a powerful tool for modeling computation in a way that is both efficient and expressive. The beauty lies in the locality and parallelism. Each interaction only involves two agents and their immediate connections. This means no global state, no shared memory, no synchronization needed. Any two independent interactions can happen simultaneously. This makes interaction combinators an ideal foundation for massively parallel computation.

2.2. Interaction Calculus

Interaction calculus is a higher-level language that maps onto interaction nets developed by HigherOrderCo. It is inspired by lambda calculus but adapted to the interaction net model. In interaction calculus, terms are represented as graphs and computation is performed through graph rewriting rules similar to those in interaction nets. Interaction calculus introduces constructs for defining functions, applying functions to arguments and managing resources in a way that aligns with the principles of interaction nets and linear logic.

3. Soma Language Design

Soma is inspired by Lean in the sense that it strives to have a minimal core language with universe polymorphism, dependent types and a powerful type system.

Due to this similarity and for convenience, we'll use Lean's syntax to illustrate some of the features of Soma, but the final syntax is bound to differ.

4. Memory Management

In this section, we formalize the memory management strategy employed by Soma. The system achieves deterministic, GC-free memory management through two primitives derived from interaction net semantics:

- **DUP** (duplication): produces two independently owned copies of a value.
- **ERA** (erasure): destroys a value and frees its resources.

These primitives are inserted automatically by the compiler based on static usage analysis. No programmer annotation is required.

4.1. The Duplication Problem

In a functional language without mutable state, every value is logically immutable. When a value is used in multiple places, the language must either share a single representation (as in garbage-collected languages) or produce independent copies (as in linear or affine systems). Soma takes the latter approach: every use of a value consumes it and multiple uses require explicit duplication.

Consider a simple example:

```
let x = expensive_computation in (f x, g x)
```

The variable x is used twice. The compiler transforms this into:

```
let x = expensive_computation in
let (x1, x2) = DUP(x) in
(f x1, g x2)
```

Each x_i is an independently owned copy consumed exactly once. The original x no longer exists after the DUP.

The problem becomes non-trivial in the presence of branching:

```
let x = expensive_computation in
case some_function x of
  Left y => (f y, g x)
  Right z => (h x, g x)
```

Here x is used once in the Left branch and twice in the Right branch. A naive approach that sums uses across branches overestimates the required copies. Before addressing lazy duplication (Section 4.5), we must first formalize the placement of DUP and ERA nodes.

4.2. Usage Counting and DUP Placement

The compiler must determine, for each binding, where to insert DUP and ERA nodes so that every value is consumed exactly once on every execution path. We consider three progressively refined strategies to motivate the final design.

4.2.1. Naive Approach: Additive Counting

The simplest approach sums all uses of a variable across all branches:

```
let x = expensive_thing in
if cond then
  work x x      -- 2 uses of x
else
  work2 x       -- 1 use of x
```

Additive counting yields 3 total uses, causing the compiler to emit 2 DUP nodes at the binding site of x , producing 3 copies. In the else branch, 2 of the 3 copies are unused and must be explicitly erased. This is correct but wasteful. The actual maximum simultaneous need is 2 (from the then branch).

4.2.2. Binding-Site Placement with Max-Counting

A better approach counts usage **per branch** and takes the maximum:

$$u_{\text{branch}(x)} = \max_{i \in \{1, \dots, n\}} u_i(x)$$
$$u_{\text{total}(x)} = u_{\text{outer}(x)} + u_{\text{branch}(x)}$$

This reduces the number of DUP nodes at the binding site to $u_{\text{total}(x)} - 1$ and composes naturally with nested branches. However, it still produces unnecessary ERAs: on branches that use fewer than max copies, the excess copies must be erased.

4.2.3. Split-Site DUP Placement

The optimal approach is to insert DUP nodes at **split sites**, points in the program where a value's ownership must diverge between independent continuations, rather than concentrating them at the binding site.

When a variable x is live across a branch point and also used outside that branch, the compiler emits one DUP before the branch: one output serves the outer uses and the other enters whichever branch executes. Each branch then handles its own internal DUPs independently.

Consider a concrete example:

```
let x = ... in
f x                -- 1 outer use
if cond1 then
  if cond2 then
    g x x x        -- 3 inner uses
  else
    h x            -- 1 inner use
else
  42               -- 0 inner uses
```

Under split-site placement, this becomes:

```
let x = ... in
let (x_f, x_branch) = DUP(x) in
f x_f
if cond1 then
  if cond2 then
    let (x1, x2) = DUP(x_branch) in
    let (x11, x12) = DUP(x1) in
    g x11 x12 x2
  else
    h x_branch
else
  ERA(x_branch)
  42
```

The per-path costs are:

Path	DUPs	ERAs	Uses of x
cond1 = T, cond2 = T	3	0	4
cond1 = T, cond2 = F	1	0	2
cond1 = F	1	1	1

On every path, the number of DUP nodes is exactly $u_{\text{path}(x)} - 1$ where $u_{\text{path}(x)}$ is the number of actual consumptions of x on that path. The only waste is a single DUP + ERA pair on the cond1 = F path, where one copy enters the branch but is immediately erased because that branch does not use x .

This waste is unavoidable without duplicating the outer code ($f\ x$) into each branch, a code-motion optimization that is semantically valid in a pure language but causes exponential code size growth with nesting depth. In practice, the single unnecessary ERA is negligible: for Tier 1 values ERA is a no-op, for Tier 2 it is one free and for Tier 3 the DUP-ERA annihilation rule eliminates both the DUP and ERA in $O(1)$ with zero copies made.

We adopt split-site placement as Soma's DUP placement strategy.

Definition. For a binding $\text{let } x = e \text{ in body}$, DUP nodes are inserted at points where x 's ownership must diverge between independent continuations. The number of DUP nodes on any execution path for binding x is exactly $u_{\text{path}(x)} - 1$, where $u_{\text{path}(x)}$ is the number of consumptions of x on that path. On paths where x enters a branch but is not used, a single ERA node reclaims the unused copy.

Of course, this requires the compiler to be aware of the control flow structure of the program. Therefore, we can't use Lean 4's approach to pattern matching through higher-order functions in their core calculus which obscures the control flow graph.

4.2.4. Higher-Order and Recursive Usage

When a value is passed as an argument to a function, the caller transfers ownership. The callee receives exactly one owned copy. If the callee uses the argument multiple times, its own DUP chain handles the duplication, this is determined by the callee's own usage analysis, not the caller's.

For recursive functions, usage at each call site is always finite (one owned copy per argument per call). The recursion itself does not create unbounded static usage counts; each invocation independently manages its own arguments through DUP chains computed from the function body's usage analysis.

4.3. Ownership Rule

The duplication and erasure strategy is grounded in a single rule:

Every value has exactly one owner at every point in the program.

A DUP node takes one owned value and produces two independently owned values, the original ceases to exist as a distinct entity. An ERA node takes one owned value and destroys it. Every wire in the interaction net carries exactly one owned value.

This invariant is a direct consequence of the interaction net model. Lafont's interaction nets (1990) require that every port is connected to exactly one other port; this one-to-one wiring ensures unique ownership by construction.

The consequences are:

1. No reference counting is required for the common case.
2. No garbage collector is needed.
3. Deallocation is deterministic: every value is freed at a statically known program point (either at its last use, or at an ERA node on branches that do not use it).
4. Use-after-free is impossible: once a value is erased, no wire in the net references it.

5. Cyclic data structures cannot arise: the interaction net is a directed acyclic graph by construction, since every wire connects exactly two ports with no sharing.

4.4. Tiered Duplication Strategy

Not all values require the same duplication mechanism. The compiler selects a strategy at compile time based on the type of the value being duplicated.

4.4.1. Tier 1: Flat Types

For integers, booleans, floats, characters and small structs that fit in machine registers, DUP is a register copy and ERA is a no-op (no heap allocation to free). These values are represented as tagged words in the runtime: the low 3 bits encode the type tag and the remaining bits hold the payload. This has zero overhead, identical cost to Rust's Copy semantics.

4.4.2. Tier 2: Fixed-Size Heap Objects

Closures, records and fixed-size structures with heap-allocated fields require allocation on the heap. The compiler generates type-specialized clone and drop functions at monomorphization time.

DUP allocates a new header, copies the fields and recursively clones any heap-allocated sub-fields. ERA frees the header and recursively erases sub-fields. The compiler knows the exact layout at monomorphization time.

Closures are a particularly important case. A closure is a heap-allocated object containing a function pointer, an arity and an array of captured environment values. DUP on a closure allocates a new closure header and recursively clones any pointer-typed environment slots. ERA frees the environment slots and the closure header. The runtime uses per-thread memory pools with size-class allocation (small closures ≤ 48 bytes, medium ≤ 112 bytes, large via `malloc`) to minimize allocation overhead.

This achieves the same cost model as a Rust Clone/Drop implementation, but requires zero programmer annotation.

4.4.3. Tier 3: Lazy Duplication via Superposition Nodes

For recursive data structures (lists, trees and any inductively defined type), eager duplication requires traversing an unbounded structure. A naive DUP on a list of N elements is $O(N)$ regardless of how much of the list each consumer actually accesses. This is where Soma's interaction net semantics enable a fundamentally more efficient strategy.

When DUP is applied to a value of recursive type, instead of performing a deep copy, the runtime creates a **superposition node** (SUP). A SUP represents a value that has been logically duplicated but whose copies have not yet been physically separated. Each consumer receives a reference that may point to either a real constructor or a SUP node.

This design follows the symmetric interaction combinators of Lafont (1997), using the same agent types $(\gamma, \delta, \varepsilon)$ and interaction rules.

4.4.3.1. Labeling and Scope Management

Each DUP node in the compiled program carries a **label**, which is a natural number assigned statically at the DUP's source site. The label determines how DUP and SUP nodes interact. Matching labels trigger annihilation (the common case), while different labels trigger commutation (handling nested duplications).

Crucially, Soma does not employ an oracle. Lamping's original optimal reduction algorithm (1990) required an oracle in order to handle arbitrary untyped λ -terms. The oracle nodes accumulate during reduction and can cause exponential overhead, undermining the optimality claim in practice.

Soma avoids this problem entirely. In Soma's compiled output, every DUP node is emitted by the compiler at a known source location with a statically assigned label. The label assignment is deterministic: each syntactic DUP site receives a unique label at compile time. There is no dynamic label generation and no runtime scope tracking. When a DUP meets a SUP, the interaction is fully determined by comparing the two labels which is a single integer comparison, not a graph traversal.

The pathological self-copying terms that require the oracle are not expressible in Soma's type system. A well-typed term in Soma can duplicate values (via the DUP mechanism), but the structure of the duplication is always statically determined by the type checker's usage analysis. There is no mechanism by which a term can receive an opaque value and duplicate it in a way that the compiler did not anticipate.

4.4.3.2. Interaction Rules

The behavior of SUP nodes is governed by five interaction rules:

DUP-Constructor commutation. When a consumer accesses a SUP and finds a constructor underneath, the DUP pushes through the constructor: two new constructor cells are allocated (one per consumer) and each field is wrapped in a fresh SUP node rather than being copied. Duplication is deferred to the fields.

$$\text{DUP}^l(\text{Cons}(h, t)) \Rightarrow (\text{Cons}(\text{SUP}^l(h)_0, \text{SUP}^l(t)_0), \text{Cons}(\text{SUP}^l(h)_1, \text{SUP}^l(t)_1))$$

In simple terms, this is just the commutation rule of interaction combinators: the DUP and the constructor swap places, with the constructor duplicating the DUP's label and pushing it down to the fields.

DUP-ERA annihilation. When one consumer erases its copy, the DUP is eliminated entirely and the surviving consumer receives the original value. No copy is ever made. This is the key rule that makes defensive duplication (DUP at binding sites in the presence of branches) essentially free: if only one branch executes, the other branch's ERA annihilates the DUP.

SUP-ERA erasure. When an ERA meets a SUP node, the SUP is freed and both values inside it are recursively erased. This handles the case where a duplicated value is no longer needed by either consumer.

SUP-DUP same-label annihilation. When a DUP with label l encounters a SUP with the same label l , they cancel out. The two values inside the SUP are distributed directly to the

two consumers. This is the common case: a value duplicated once, then both copies accessed costs $O(1)$.

$$\text{DUP}^l(\text{SUP}^l(a, b)) \Rightarrow (a, b)$$

SUP-DUP different-label commutation. When a DUP with label l_1 encounters a SUP with a different label l_2 , both push through each other, creating new SUP and DUP nodes with their respective labels preserved. This handles nested duplications correctly without oracle machinery.

$$\text{DUP}^{l_1}(\text{SUP}^{l_2}(a, b)) \Rightarrow (\text{SUP}^{l_2}(\text{DUP}^{l_1}(a)_0, \text{DUP}^{l_1}(b)_0), \text{SUP}^{l_2}(\text{DUP}^{l_1}(a)_1, \text{DUP}^{l_1}(b)_1))$$

4.4.3.3. Access-Site Code Generation

When compiled code accesses a value of recursive type (pattern match, field projection), the compiler emits a SUP check before the access.

This is one branch per access. The common case (the value is a real constructor, not a SUP) is highly predictable by modern branch predictors. After resolution, subsequent accesses hit the real value directly.

4.4.3.4. Asymptotic Advantage

Consider a list of N elements that is duplicated, where one consumer accesses only the first K elements:

Operation	Eager clone	Soma Tier 3
Clone list, both use all N	$O(N)$	$O(N)$ (deferred)
Clone list, one uses $K < N$	$O(N)$	$O(K)$
Clone list, one side erased	$O(N)$	$O(1)$

The savings grow with the size of the data structure and the asymmetry of access patterns between consumers. In the common case where a value is duplicated defensively (because it appears in multiple branches) but only one branch executes, the DUP-ERA annihilation rule ensures zero copies are made.

4.4.3.5. Completeness of the Interaction Rules

We claim that the five rules above are complete for well-typed Soma programs. Every active pair (two agents whose principal ports are connected) has an applicable rule. The possible active pairs involving DUP, ERA, SUP and constructor agents are:

- $\text{DUP} \leftrightarrow \text{Constructor}$: handled by DUP-Constructor commutation.
- $\text{DUP} \leftrightarrow \text{ERA}$: handled by DUP-ERA annihilation.
- $\text{DUP} \leftrightarrow \text{SUP}$ (same label): handled by same-label annihilation.
- $\text{DUP} \leftrightarrow \text{SUP}$ (different label): handled by different-label commutation.
- $\text{ERA} \leftrightarrow \text{Constructor}$: standard interaction net erasure (ERA pushes through the constructor, erasing each field).
- $\text{ERA} \leftrightarrow \text{SUP}$: handled by SUP-ERA erasure.
- $\text{Constructor} \leftrightarrow \text{Constructor}$ (same type): standard annihilation.

- $SUP \leftrightarrow SUP$: cannot form an active pair. A SUP's principal port connects to a consumer (a DUP, ERA, or a pattern match), not to another SUP.

No other active pairs arise in well-typed programs. In particular, $DUP \leftrightarrow DUP$ does not occur because DUP nodes are always separated by the values they are duplicating.

4.5. Multithreading Considerations

Soma's runtime employs a work-stealing scheduler with per-thread Chase-Lev dequeues. When a duplicated value is consumed by different threads across a fork/join boundary, the SUP node becomes shared memory. Resolution of a SUP by two threads simultaneously would constitute a data race.

Soma adopts a thread-local SUP policy: when forking a task that captures one side of a duplicated recursive value, the value is eagerly copied at the fork boundary (falling back to Tier 2 behavior). SUP nodes exist only within a single thread's subgraph. This avoids all contention since each worker operates on independently owned data.

This policy is simple and predictable. It can be refined in the future if profiling reveals that cross-thread duplication of large recursive structures is a bottleneck. Alternative strategies include atomic CAS-based SUP resolution (adding one atomic operation per cross-thread SUP resolution) and ownership transfer at fork boundaries (the parent gives up its side of the SUP to the child, ensuring each thread owns exactly one side with no sharing).

4.6. Comparison with HVM and Bend

HigherOrderCo's HVM and its surface language Bend represent an alternative approach to applying interaction net semantics to functional programming. Both Soma and Bend use interaction combinators as their computational foundation and support dependent types, but their architectures differ fundamentally in the relationship between interaction nets and the execution model.

4.6.1. Runtime Interpretation vs. Ahead-of-Time Compilation

HVM is an **interaction net runtime**. Programs are represented as graphs of agents in memory and execution proceeds by graph rewriting: the runtime scans for active pairs (two agents whose principal ports are connected) and applies the corresponding interaction rule. Every value including integers, booleans and function pointers is a node in the interaction net with ports and pointers.

Soma uses interaction nets as a **compilation model**. The DUP/ERA/SUP semantics inform the compiler's code generation, but the output is flat, imperative LLVM IR. Tier 1 values are register copies with no heap representation. Tier 2 values use compiler-generated clone and drop functions that compile to ordinary function calls. Only Tier 3 values (recursive data structures with SUP nodes) retain interaction-net-like behavior at runtime and even then the runtime representation is a tagged union check, not graph rewriting.

The consequence is that Soma eliminates the overhead of the graph representation itself. In HVM, every non-trivial value is a heap-allocated node with port pointers, which incurs allocation overhead, pointer indirection and cache pressure on every operation. Soma's Tier 1

and Tier 2 values have zero interaction net overhead at runtime and generate the same machine code that a conventional compiled functional language would produce.

4.6.2. Optimality and the Lazy Duplication Trade-off

HVM claims Lévy-optimality: it never duplicates a redex. When a function body is shared by two consumers, an optimal reducer reduces it once and distributes the result through the sharing node, rather than copying the body and reducing it independently in each copy.

Soma is **not** Lévy-optimal. The tiered duplication strategy sacrifices optimality for predictable performance:

- Tier 1 (flat types): DUP is a register copy. No redex can exist in a flat value, so no sharing opportunity is lost.
- Tier 2 (closures, records): DUP eagerly deep-copies the value. If a closure captures an unreduced computation, both copies will evaluate it independently. An optimal reducer would have shared the computation.
- Tier 3 (recursive data): DUP is lazy via SUP nodes, recovering optimal behavior for data traversal patterns.

Consider the following example:

```
let f = \x -> expensive x in
let (f1, f2) = DUP(f) in
(f1 arg, f2 arg)
```

In HVM, *f* is a net node. DUP creates a sharing node. When *f*₁ is applied, the body of the lambda begins reducing. If *f*₂ is applied to the same argument, the result is shared and *expensive* is computed once.

In Soma, *f* is a Tier 2 closure. DUP allocates a new closure and copies the captured environment. The two closures are independent. *expensive* is computed twice.

This is a deliberate trade-off. Lévy-optimality minimizes the number of β -reduction steps, but each step in HVM involves pointer chasing through a heap-allocated graph with associated cache misses and allocation overhead. Soma's eager copy of a closure is a small memcpy followed by native code execution with full register allocation and branch prediction. For the vast majority of programs, the constant-factor advantage of native code execution dominates the asymptotic advantage of optimal sharing.

The cases where optimality provides a genuine asymptotic advantage are rare in practice. Soma's Tier 3 lazy duplication captures the case where laziness matters most in real programs: large data structures where consumers have asymmetric access patterns.

4.6.3. Parallelism Model

Because HVM maintains the interaction net as a runtime data structure, it can exploit parallelism at the granularity of individual interactions. Any two independent active pairs in the net can reduce simultaneously. This enables automatic, fine-grained parallelism without programmer annotation, including GPU execution, where thousands of interactions proceed in parallel.

The trade-off is: HVM achieves pervasive parallelism at the cost of graph representation overhead. Soma achieves better single-thread performance and cache behavior at the cost of less automatic parallelism.

4.6.4. Summary

Property	HVM / Bend	Soma
Execution model	Runtime graph rewriting	Ahead-of-time LLVM compilation
Value representation	All values are net nodes	Tagged words / native structs
Flat type overhead	Node allocation + ports	Zero (register copy)
Closure duplication	Lazy (optimal)	Eager (Tier 2 clone)
Recursive data duplication	Lazy (SUP nodes)	Lazy (SUP nodes)
Lévy-optimality	Yes	No (Tier 2 is eager)
Parallelism	Automatic, fine-grained (GPU)	Explicit fork/join (CPU)
Cache behavior	Poor (pointer-heavy graph)	Good (native data layout)
Single-thread performance	Lower (interpretation overhead)	Higher (native code)